Chrysostomos Koubanis

4325

Technical Report: Next Digit Generation using CNNs (Phase 1)

1. Introduction

The objective of this assignment was to design and train a Convolutional Neural Network (CNN) capable of learning the semantic sequence of numbers. Specifically, the model accepts an image of a handwritten digit **N** (from the MNIST dataset) as input and generates the image of the next digit in the sequence **(N+1)**.

This report details **Phase 1**, where the problem was approached as an **Image-to-Image Regression** task using an Encoder-Decoder architecture.

2. Code Structure & Implementation Logic

The project was modularized into three distinct files to separate data handling, model architecture, and training logic.

A. data_loader.py (Data Preparation)

This file handles the loading and preprocessing of the MNIST dataset.

- **Key Function:** create_pairs(x, y)
    - **Logic:** This function is critical for the assignment. It groups the MNIST images by their digit class (0-9). It then iterates through the numbers 0 to 9, pairing every image of number N with an image of number (N+1) % 10.
    - **Result:** It returns two numpy arrays: x_paired (Input: Image of 3) and y_paired (Target: Image of 4). This transforms the problem from classification to transformation.

Normalization: All pixel values are scaled to the range [0, 1] to ensure stable gradient descent.

B. model.py (The Architecture)

This file defines the **Encoder-Decoder** architecture. Below is the structure of the final model used in this phase:

1. **Encoder (Compression):**
    - **Input:** 28x28 Grayscale Image.
    - **Layer 1:** Conv2D (32 filters, 3x3 kernel, stride=2). Downsamples to 14x14.
    - **Layer 2:** Conv2D (64 filters, 3x3 kernel, stride=2). Downsamples to 7x7.
    - **Activation:** ReLU is used after each convolution to introduce non-linearity.
    - **Normalization:** BatchNormalization is applied to stabilize training.
2. **Decoder (Reconstruction):**
    - **Layer 3:** Conv2DTranspose (64 filters, stride=2). Upsamples to 14x14.
    - **Layer 4:** Conv2DTranspose (32 filters, stride=2). Upsamples to 28x28.
    - **Output Layer:** Conv2D (1 filter, sigmoid activation). Compresses values to [0, 1] to represent pixel intensity.

C. train.py (Training & Visualization)

This script functions as the training environment for the model. It saves the model as next_digit_generator.keras and it contains the train_network() function, which orchestrates the entire learning process.

- **Hyperparameters:**

    - **Epochs:** Set to 20. We observed that training beyond 20 epochs yielded diminishing returns for this specific architecture, with no significant gain in accuracy.
    - **Batch Size:** Set to 128. This size provides an optimal balance between memory efficiency and gradient estimation accuracy.
    - **Optimizer:** Adam (Adaptive Moment Estimation) was selected to ensure efficient convergence during optimization.
- Visualization (The Contrast Filter):

  Inside the visualization block of this file, we implemented a specific post-processing technique: sharpened = prediction^2. It is important to note that this operation is not part of the neural network architecture itself; rather, it serves as a visual aid during testing. By squaring the pixel values, we push gray pixels closer to black, artificially increasing the contrast to make the blurry output more readable for evaluation.

3. Experimental Log

Below is the chronological log of experiments conducted and the findings derived from each iteration.

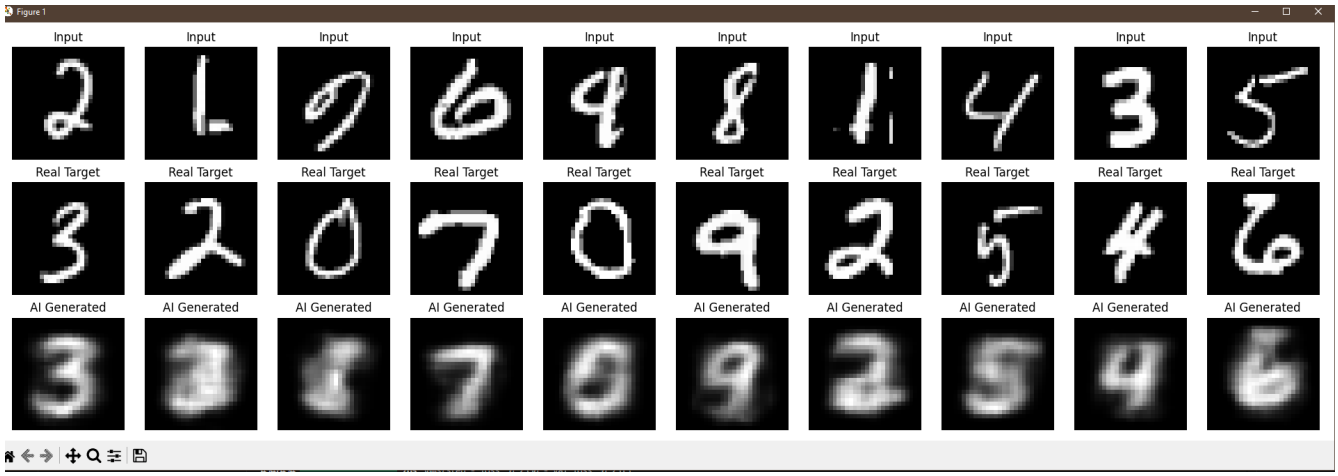
Experiment 1: Baseline Model

- **Configuration:**

  Loss Function = Binary Crossentropy

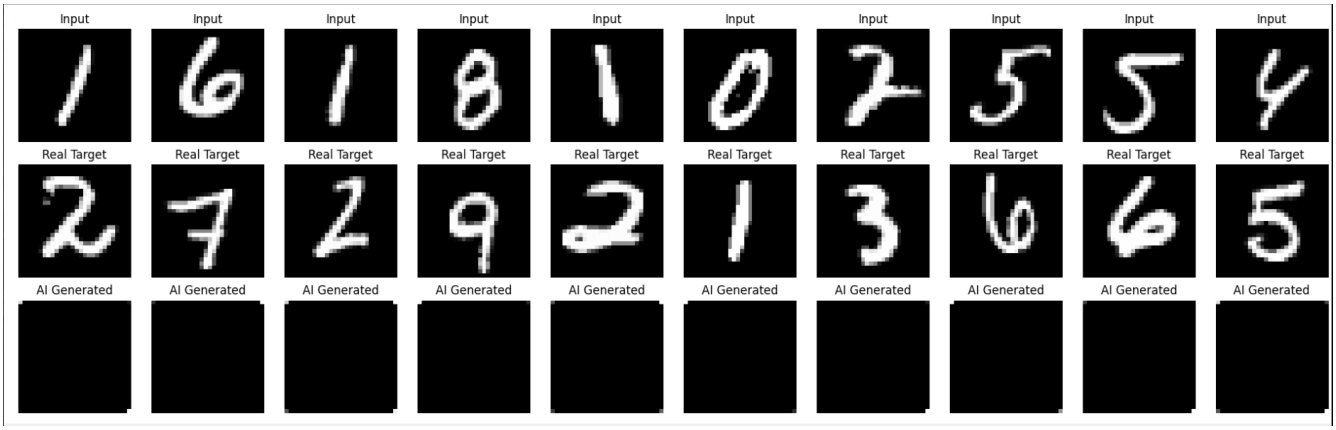
  Optimizer = Adam

- **Observation:** The output images were correct in shape but extremely blurry.
- **Analysis:** This is due to **Regression to the Mean**. The input "3" maps to thousands of different styles of "4"s in the dataset. Minimizing the loss causes the model to predict the statistical average of all possible targets, resulting in a blurry image.



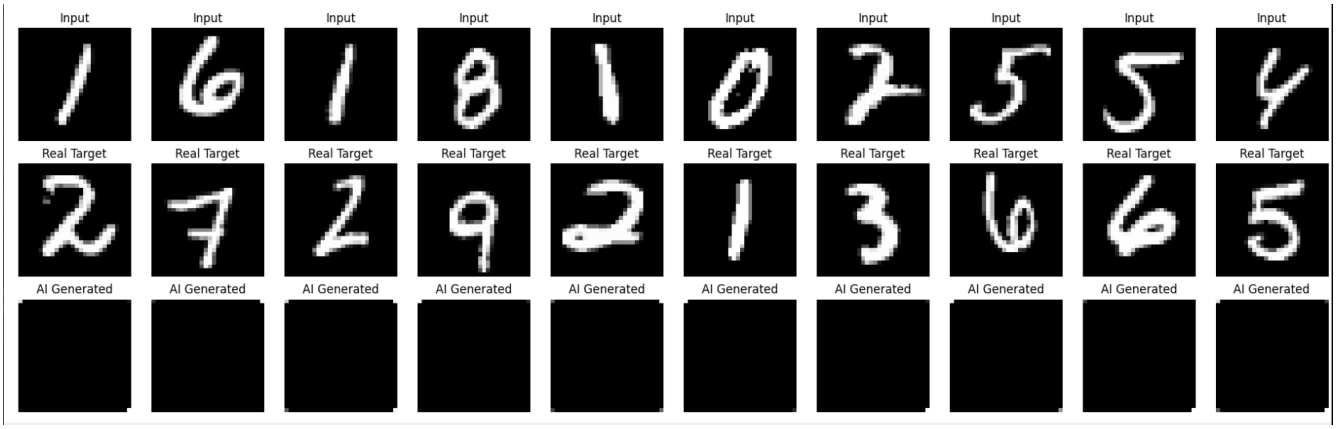
Experiment 2: Changing Loss Function

- **Change:** Switched the loss function from Binary Crossentropy to **MAE (Mean Absolute Error)**.
- **Hypothesis:** Binary Crossentropy punishes incorrect pixels heavily. We hypothesized that MAE would punish small gray errors less, potentially encouraging the model to be "bolder" and create sharper transitions.
- **Result: Failure.** The model output solid black images. The loss stagnated at ~0.1127
- **Analysis:** This is known as **Background Collapse**. Since ~90% of an MNIST image is black background, the model learned that predicting "all black" yields a 90% accuracy score with zero effort.



Experiment 3: Increasing Punishment

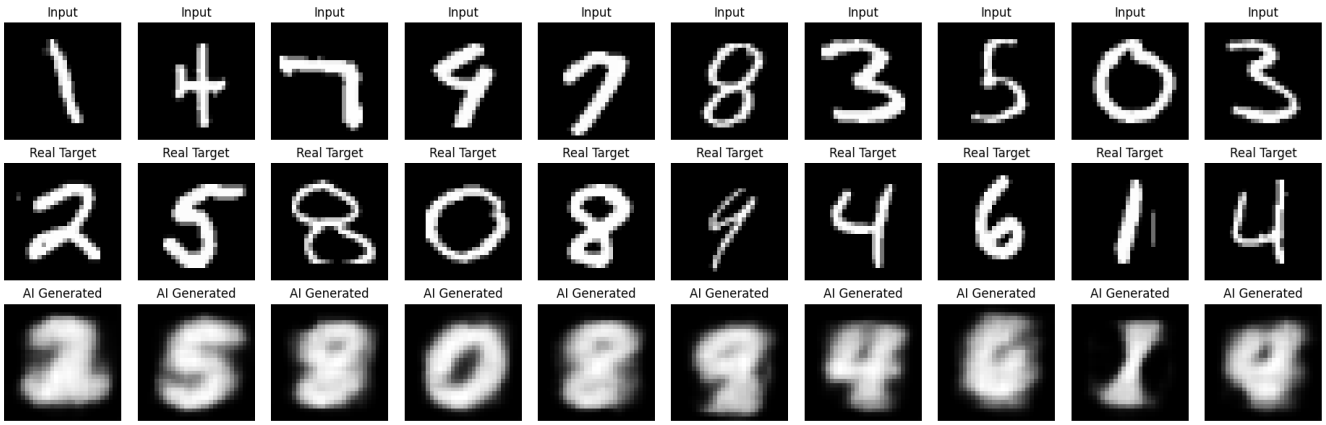
- **Change:** Switched Loss Function to MSE (Mean Squared Error).
- **Hypothesis:** I wanted to see if the output would be better than MAE.
- **Result: Failure.** The outputs remained black. The loss stagnated at ~0.1127.
- **Analysis:** The penalty for missing the white pixels was still not high enough to overcome the easy local minimum of predicting the background.



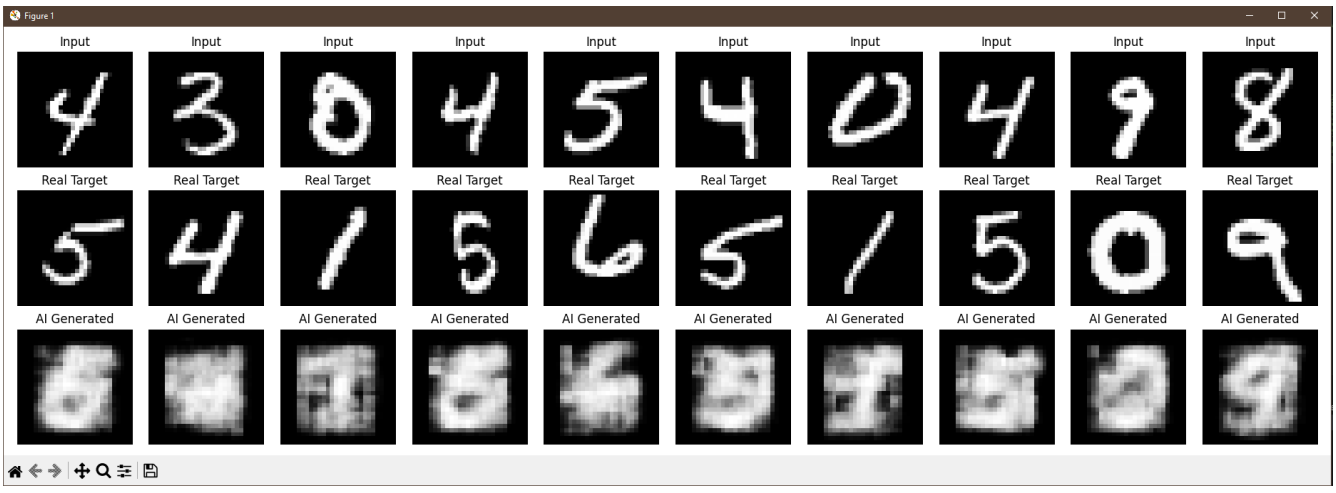
Experiment 4: Custom Loss Function

- **Change:** Implemented a custom weighted loss function(first MSE ,then binary crossentropy). The goal was to manually multiply the error on white pixels by a factor 10 to force the model to draw.
- **Result: Regression.** The outputs were no longer black, but they were significantly **more blurry** than Experiment 1.
- **Analysis:** The custom loss destabilized the gradients, causing the model to be "unsure" and spread probability mass over a wider area, increasing blur.Also the binary crossentropy custom loss function did significantly worse than the MSE one.

MSE Custom Loss Function

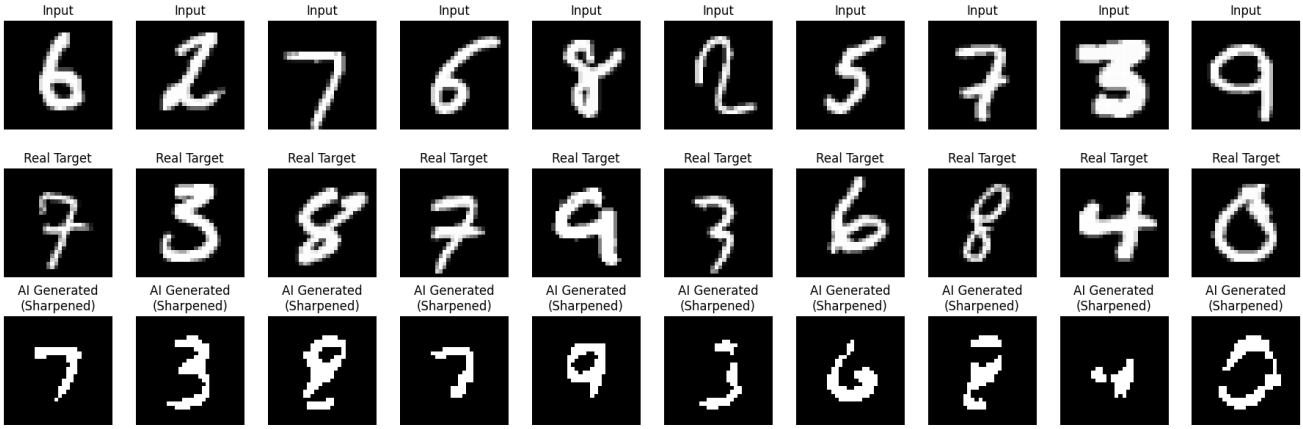


Binary Crossestropy Custom Loss Function



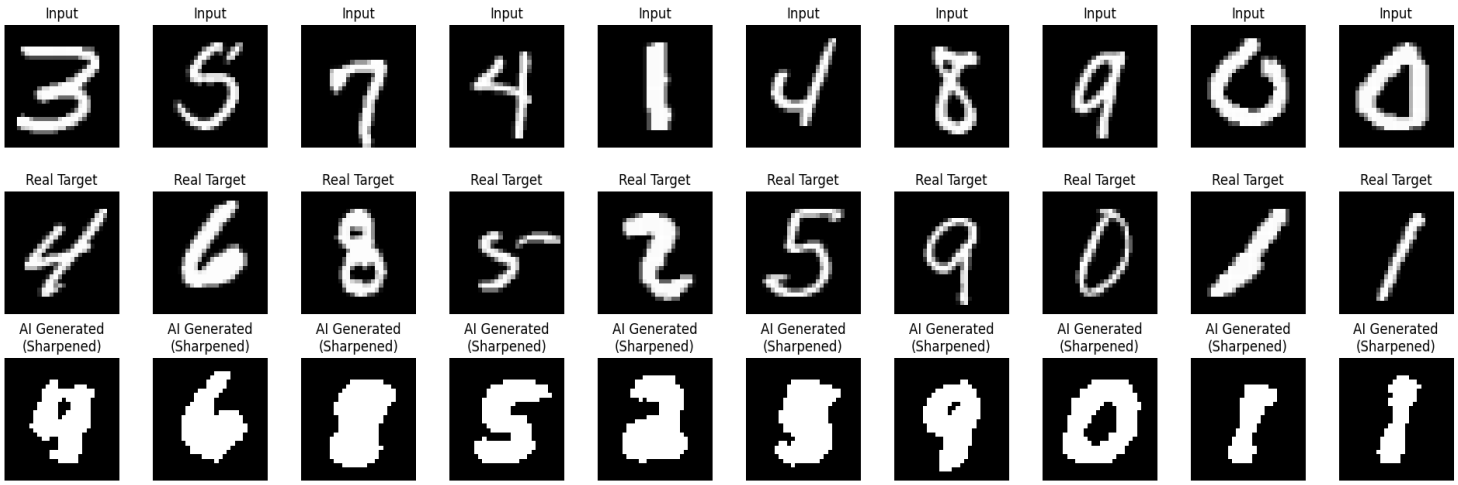
Experiment 5: Post-Processing Threshold

- **Change:** Reverted to Binary Crossentropy but added a "Sharpness Filter" during visualization. Pixel intensities exceeding 0.5 were mapped to white, while all others were mapped to black.
- **Result:** The blur was gone, but the digits were **fragmented**.
- **Analysis:** The 0.5 threshold was too strict, deleting valid parts of the digit.



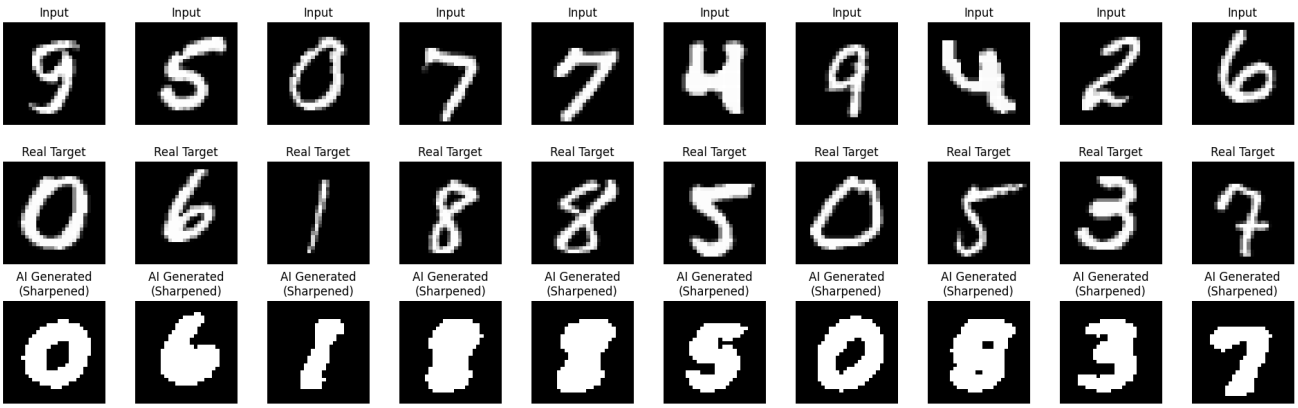
Experiment 6: Structural Changes

- **Change:** Added **BatchNormalization** layers and lowered the visualization threshold filter to 0.2.
- **Result:** Connectivity improved, but the images looked "Blocky" or "Lego-like".
- **Analysis:** This artifact was caused by the UpSampling2D layer used in the Decoder. By simply duplicating pixels to scale up, it created a jagged, pixelated look.



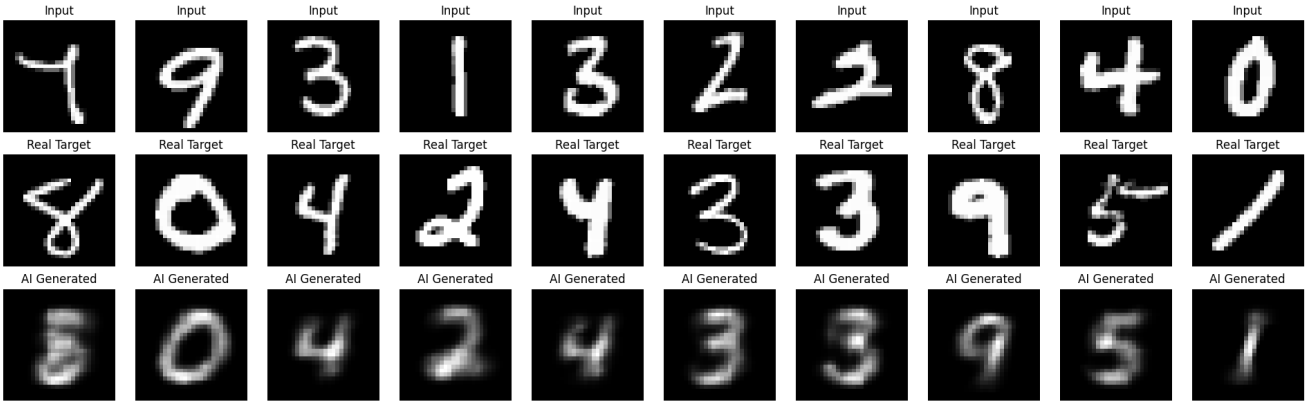
Experiment 7: Smart Upscaling

- **Change:** Replaced all UpSampling2D layers with **Conv2DTranspose** (Stride=2).
- **Result:** Almost no visual change



Experiment 8: Contrast Boosting

- **Change:** Instead of a hard threshold, we applied a soft contrast boost (prediction ** 2).
- **Result:** This yielded the best results for the Encoder-Decoder architecture. The digits were recognizable and structurally sound, though inherent blurriness remained.
- **Training Duration:** ≈ 41 minutes



4. Phase 1 Conclusion

Through this rigorous experimentation process, we successfully optimized the Convolutional Encoder-Decoder architecture to its theoretical limits. We achieved significant structural improvements through specific engineering decisions:

- **Geometric Stability:** Replacing UpSampling2D with Conv2DTranspose eliminated "blocky" artifacts and allowed the model to learn smoother upscaling features.
- **Signal Integrity:** Introducing BatchNormalization ensured better connectivity in the digit strokes by stabilizing the gradient flow through deep layers.
- **Visual Clarity:** Implementing a custom contrast boosting technique (prediction ** 2) helped mitigate the inherent "grayness" of the output.

However, despite these architectural optimizations, the generated digits remained inherently **blurry** and lacked the sharp, binary texture of real ink.

The fundamental limitation lies in the nature of **Pixel-wise Loss Functions** (whether Binary Crossentropy, MSE, or MAE).

- In **Binary Crossentropy** (which we used for the majority of successful tests), the model is penalized for every individual pixel that differs from the ground truth.
- Since a single input (e.g., "3") can correspond to many valid output variations (e.g., a "4" that is tilted, straight, thick, or thin), the loss function forces the model to "hedge its bets."
- To minimize the penalty across all possible variations, the model predicts the **statistical average** of all potential shapes. Visually, the average of many sharp lines is a single blurry cloud.

In practice, pixel-wise losses are insufficient for multimodal image-to-image mappings like MNIST digit translation. To achieve photo-realistic sharpness, the system requires a paradigm shift: we need a loss function that evaluates the **realism of the entire image** rather than the accuracy of individual pixels. This necessitates the implementation of a **Generative Adversarial Network (GAN)** in Phase 2.

Technical Report: Phase 2 – Generative Adversarial Networks (GANs)

1) Introduction to Phase 2

To address the blurriness issues identified in Phase 1, we transitioned from standard Supervised Learning to Adversarial Learning. A Generative Adversarial Network (**GAN**) consists of two neural networks engaged in a zero-sum game:

- **The Generator**: Its objective is to create "fake" images (e.g., transforming a digit "3" into a "4") that are indistinguishable from real data. It aims to minimize the probability that the Discriminator detects the fabrication.
- **The Discriminator**: Its role is to act as a critic. It receives both "Real" images (from the dataset) and "Fake" images (from the Generator) and attempts to correctly classify them.

Resolution of the Blurring Issue:

In a GAN framework, a blurry image is readily rejected by the Discriminator because real MNIST digits are sharp, binary (black/white) shapes. To

successfully deceive the Discriminator, the Generator is compelled to make high-contrast, sharp decisions. This effectively resolves the "Regression to the Mean" problem encountered in previous phases.

2) Experimental Evolution

We use the same data_loader.py as we did in phase 1 to construct a paired dataset.

We implemented a Conditional GAN (cGAN) architecture tailored for image-to-image translation (specifically Digit N to N+1). Unlike standard GANs that generate images from random noise, our model is conditioned on an input image to strictly guide the generation process.

Run 1: The Baseline GAN

- **Architecture of the Generator:**
    - **Standard Encoder-Decoder Network**
    - **The Encoder:**
        - **2 layers (32 filters - > 64 filters)**
        - **Conv2D** with **stride=2**
        - **Batch Normalization:** Applied after every convolution to stabilize learning.
        - **ReLU:** Used as the activation function for the encoder layers.
    - **The Decoder:**
        - **2 layers (64 filters - > 32 filters)**
        - **Conv2DTranspose** with **Strides=2**.

        - **Batch Normalization** + **ReLu**
    - **Output Layer:**
        - **Conv2D with 1 filter and sigmoid activation**

- **Architecture of the Discriminator:**
    - **Downsampling (Strides vs. Pooling):**
    - Used **Conv2D with strides=2** instead of **Max Pooling** , because it allows the network to learn its own spatial downsampling operation rather than forcing a fixed pooling operation, which is standard practice in modern GANs.
    - **Filters: Layer 1: 64 filters, Layer 2: 128 filters**.
    - **Activation Function: LeakyReLU** (negative_slope=0.2) instead of standard ReLU.
    - **LeakyReLU reasoning:** Standard ReLU kills the gradient for negative inputs (the "dying ReLU" problem). In GANs, this can stop the discriminator from learning entirely. LeakyReLU allows a small gradient to flow even when inactive.
    - **Regularization: Dropout**
    - Applied **Dropout(0.3)** after every block to prevent the discriminator from becoming "overconfident" or memorizing the training data, which would overpower the generator too early.

Training Configuration (train_gan.py)

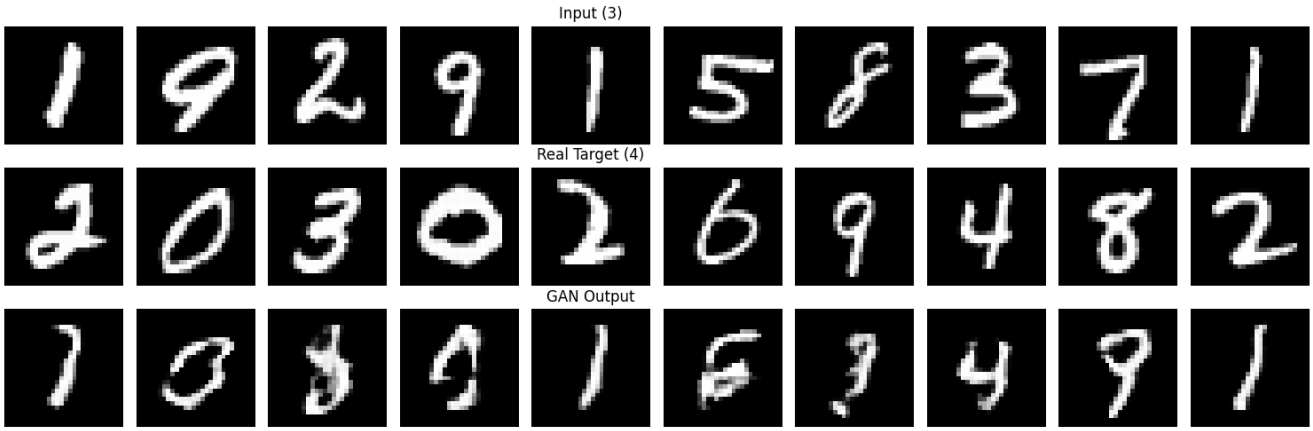
- **Optimizers (Adam):**
    - **Learning Rate:** 2e-4 (0.0002).
    - **Beta_1:** 0.5.
    - **Beta_1 value reasoning:** While standard Adam uses a Beta_1 of 0.9, GANs (specifically CGAN architectures) are known to be unstable with high momentum. Lowering it to 0.5 stabilizes training.

- **Loss Function Strategy:**
    - **Adversarial Loss:** BinaryCrossentropy. This forces the generator to create images that the discriminator thinks are real
    - **Pixel-wise Loss:** MeanAbsoluteError (L1 Loss). This measures the absolute distance between the generated pixels and the target pixels.
    - **Combined Loss:** Total Loss = Loss_GAN + (λ * Loss_L1).
- **Hyperparameters:**
    - **Lambda (λ = 100):** This dictates that the model cares **100 times more** about the image looking like the correct specific number (L1 Loss) than it cares about fooling the discriminator. This prevents the model from generating a realistic-looking "5" when it was supposed to generate a "4".
    - **Batch Size (64):** Chosen because smaller batch sizes provide noisier gradients, which can actually help the generator explore the data distribution better than large, stable batches.
- **Data Pipeline:**
    - **Buffer Size (1024):** Determines how many elements are loaded into memory for shuffling.
    - **Shuffling:** Essential to prevent the discriminator from memorizing the order of the dataset.

- **Goal:** To determine if the Adversarial approach could solve the "Blurry Output" problem identified in Phase 1.
- **Observation:** The blur was completely eliminated. The background was pitch black, and the digits were bright white.
- **Training Duration: ≈ 40 minutes**

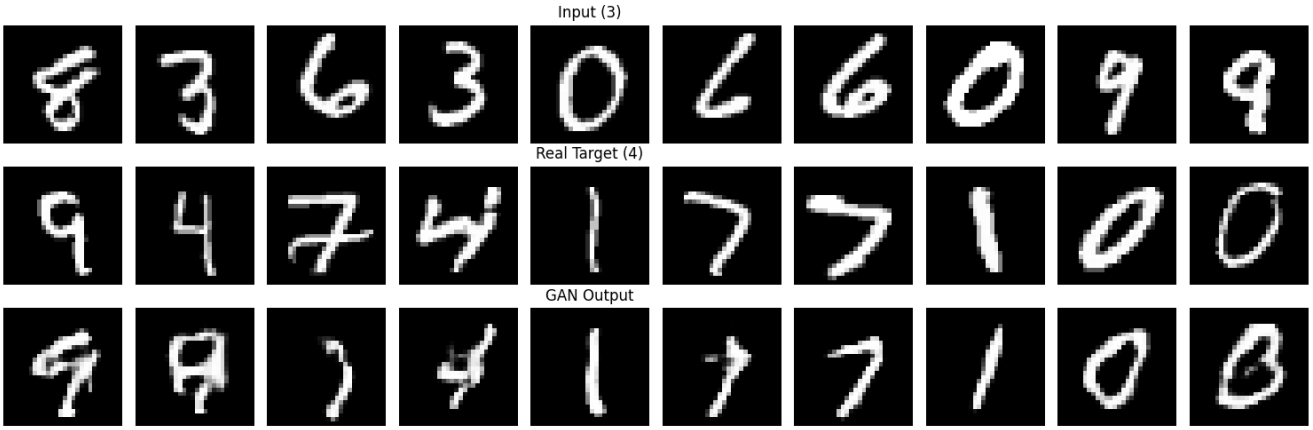
- **The Problem ("Shattered Glass"):** The digits appeared fragmented, instead of continuous strokes, the numbers resembled disconnected tiles or broken glass.
- **Analysis:** This is a known GAN failure caused by the Decoder.
- The Conv2DTranspose layer overlaps "tiles" of pixels to upscale the image.
- **Root Cause 1: Conv2DTranspose**
    - The Conv2DTranspose layer functions by projecting a "stamp" (the kernel) onto the output canvas for every input pixel.
    - Unlike standard interpolation (which smooths values), this layer adds values together where the stamps overlap. This accumulation of pixel values makes the overlap regions significantly brighter than non-overlap regions, creating a grid structure.

- **Root Cause 2: Kernel vs Stride**
    - I used a **Kernel Size of 3** with a **Stride of 2**.Because the kernel size (3) is not **divisible** by the stride (2), the overlap is uneven (3/2=1.5).That caused the generator to create a repeating pattern where some pixels are summed twice (overlap) while adjacent pixels are summed only once. This uneven "stamping" is what physically creates the shattered/checkerboard look.



Run 2: The Geometric Fix (UpSampling)

- **Change from Run 1:** We **replaced** the **Conv2DTranspose layers** with a two-step process: **UpSampling2D** followed by **Conv2D** .
- This:
    - Decouples the upscaling from the convolution, ensuring that resizing artifacts do not interfere with feature learning.
    - Fixes the kernel-stride division problem, because the Conv2D layer operates with a default **Stride of 1**. This guarantees that the kernel slides over every pixel uniformly, physically preventing any checkerboard grid from forming regardless of the kernel size used.
- **Goal:** To physically prevent the formation of checkerboard gaps by ensuring the pixel overlap is handled by a standard convolution padding='same' rather than a strided transposed convolution.
- **Observation:** The "Shattered Glass" artifacts became more rare, and the lines of the digits became smoother and more continuous, proving the geometric fix worked.
- **Training Duration: ≈ 1 hour and 20 minutes**
- **The New problem ("Melting Shapes"):** While the texture was improved , the structure of some digits was incorrect. Some digits looked "melted" or deformed
- **Analysis:** This was a **Bottleneck Issue**. Our Encoder compresses the 28x28 image down to a **7x7 feature map.** The model is forced to compress all spatial information (where the curves and lines are) into this tiny space. While it retains the general "idea" of the number (texture), it loses the precise coordinate information needed to reconstruct the exact shape, resulting in topological warping.



Note on Alternative Solutions: We could have technically fixed the checkerboard artifacts in Run 1 by changing the **Kernel Size to 4**. Since 4 is divisible by the Stride (2), the overlap would have been even. However, we chose the **Resize-Convolution** method (Run 2) because it allows us to keep using **3x3 Kernels**, which are superior for preserving the spatial "center" of features, whereas 4x4 kernels can cause image shifting.

Run 3: The Structural Fix (U-Net Architecture)

- **Change from Run 2:**
    - **Major Architecture Overhaul:** We redesigned the Generator to use a **U-Net** architecture.
    - **Skip Connections mechanism:** Instead of a straight line, we created "bridges" that connect the Encoder layers directly to the Decoder layers.We used **layers.Concatenate()** to merge the high-resolution features from the Encoder (c1, c2) with the upsampled features in the Decoder (u1, u2).This allows the model to bypass the bottleneck for spatial information. The Decoder now receives raw 28x28 and 14x14 coordinate data directly from the input.

- **Goal:** To resolve the "Melting Shapes" issue by allowing the Decoder to retain spatial details (location, scale, rotation) directly from the input image, rather than forcing the model to reconstruct everything from the compressed 7x7 latent space.
- **Observation:**
    - **Training Duration: ≈ 2 hour and 35 minutes**
    - **Success:** Structural integrity was restored. The digits maintained their proper shape and the "melting" deformation was eliminated.
    - **New Problem (Degradation):**
        - The model was trained for **30 Epochs**.

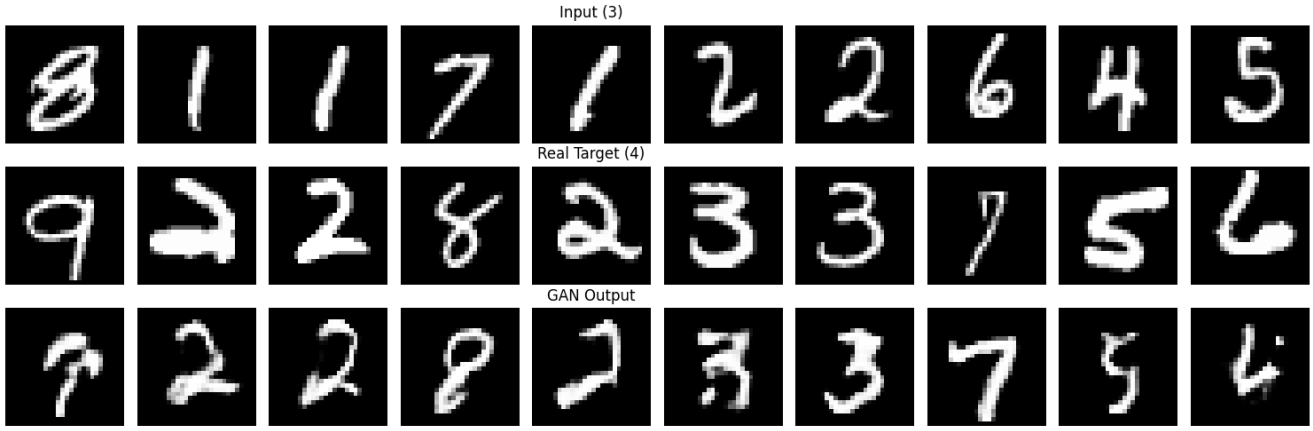
- **Visuals:** Epoch 20 produced high-quality results. However, by Epoch 30, the visual quality degraded, with the output becoming noisy.
- **Data contradiction:** Despite the visual degradation, the training logs showed stable loss values (Disc Loss ~1.0, Gen Loss ~12.5) throughout the entire run.

- **Analysis:**
  - Initially suspected as GAN Divergence, the logs disprove this theory. The losses did not explode or vanish.

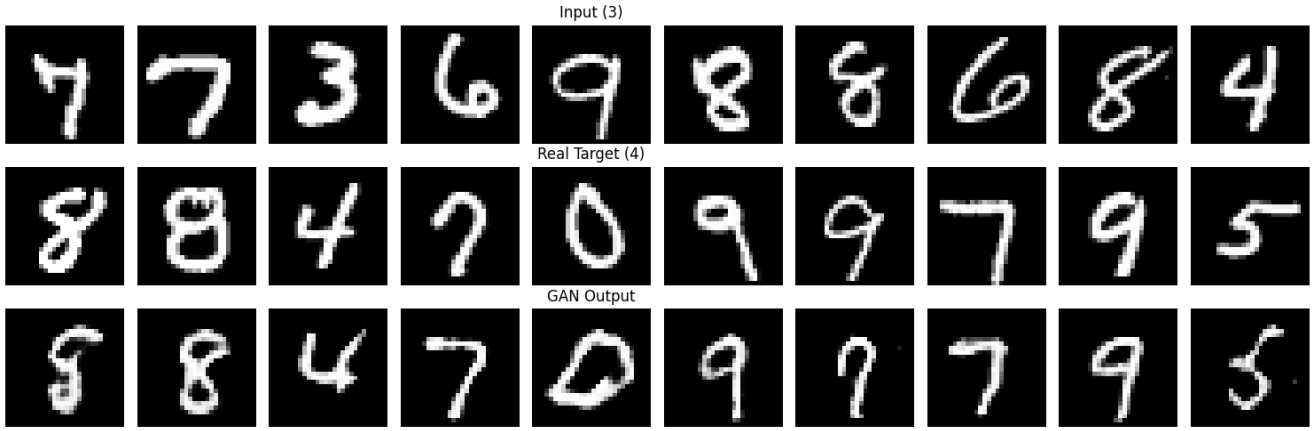
  True Cause: Training Instability (Oscillation).

  - The model likely converged to an optimal solution around Epoch 20.
  - However, without mechanisms like **Learning Rate Decay** or **Early Stopping**, the optimizer continued to make large updates, causing the model to "overshoot" the minimum and drift away from the optimal weights, introducing noise into the later epochs.

Epoch 30:



Epoch 20:



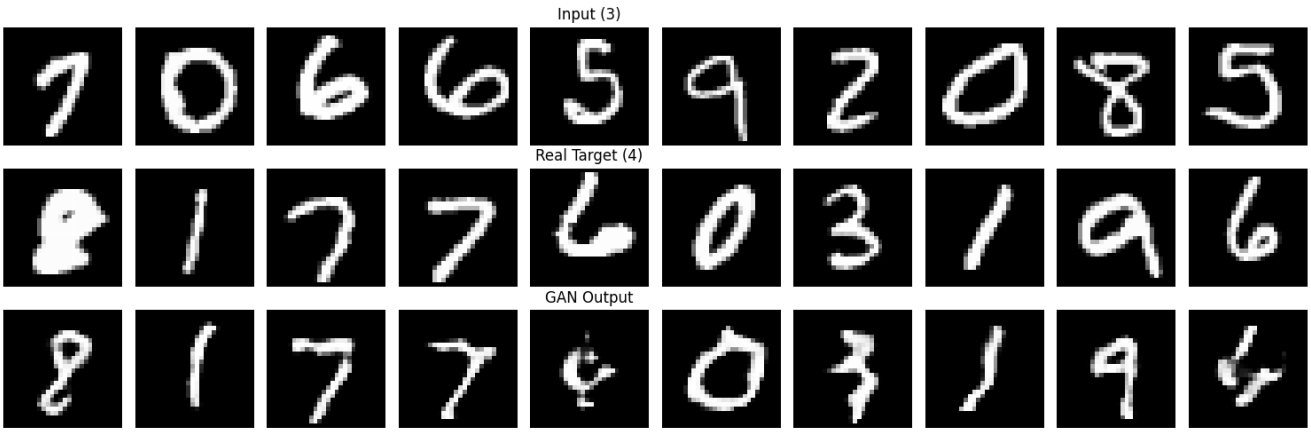
Run 4: Optimization (Early Stopping)

- **Changes from Run 3:**
    - We reduced the training duration from 30 Epochs to **20 Epochs, because based** on Run 3, the model began to oscillate and degrade after Epoch 20. We hypothesized that stopping exactly at this peak would consistently yield the best results.
    - **Interactive Verification (live_demo.py):**We developed a new script that loads the saved model (gan_generator.keras) and accepts user input (e.g., "Enter number: 3").

- **Observation**
    - **Training Duration: ≈ 1 hour and 41 minutes**
    - **Visual Results:** Surprisingly, the results at Epoch 20 were **inferior** to the results observed at gan_progress_epoch_20.png in Run 3.
    - **Live Demo Testing:**
        - While some digits transformed correctly ,others failed or produced blurry artifacts.
        - The "perfect" results seen in Run 3 were not reproducible simply by fixing the epoch count.
- **Analysis:**
    - This experiment disproved the theory that "Epoch 20 is always the sweet spot."
    - **High Variance:** The excellent performance of Run 3 (Epoch 20) was partially due to "lucky" random weight initialization.

- **Conclusion:** While the **U-Net Architecture (Run 3)** correctly solved the structural issues ("Melting"), the training process itself remains unstable.





Run 5: Discriminator Upgrade

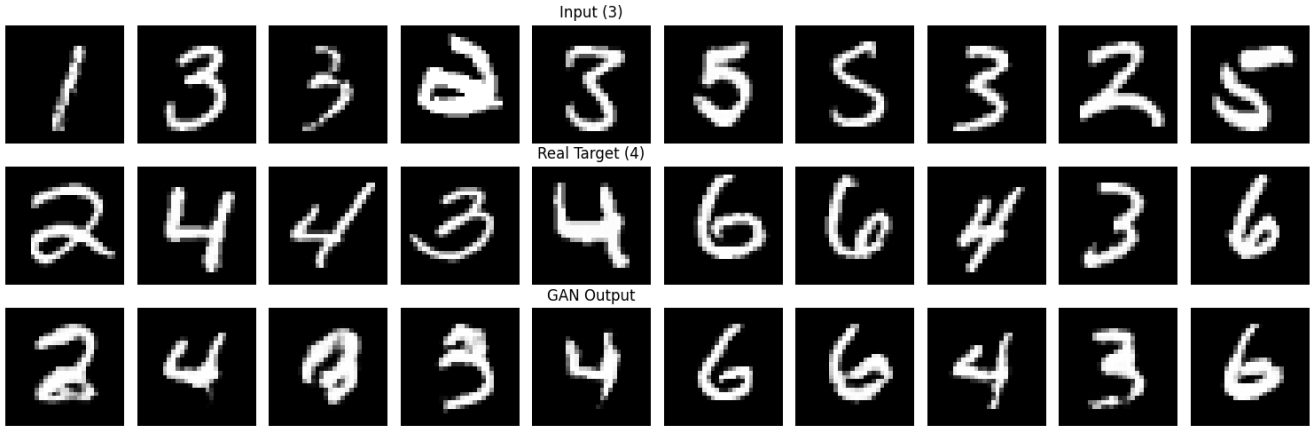
- **Changes from Run 4:**
    - **Discriminator Upgrade:** We transitioned from a standard Classifier to a **PatchGAN**. Instead of outputting a single score (Real/Fake), the Discriminator now outputs a 7*7 grid of scores, classifying local image patches as real or fake.
    - **Generator Upgrade:** We doubled the filter capacity from 32->64 to 64->128-> 256 and added **Dropout** to the bottleneck to prevent memorization.
    - **Extended Training:** Increased to 40 Epochs.
    - **Snapshotting:** Modified the training loop to save checkpoints every 10 epochs (epoch_10.keras, epoch_20.keras, etc.) and created a **compare_models.py** script to analyze the model's evolution over time.
- **Goal:** To force the Generator to focus on texture (local sharpness) rather than just global shapes, and to improve stability for longer training.
- **Observation:**
    - **Training Duration: ≈ 8 hours and 18 minutes**
    - **Mathematical Success:** The training was way more stable. The Generator Loss dropped by approximately **20%** compared to Run 4 (from ~11.3 to ~8.9), and the Discriminator maintained an equilibrium (loss ~1.2).
    - **Visual Stagnation:** Despite the mathematical improvement, the visual sharpness did not improve drastically. The images looked similar to Run 4.

- **Analysis (The "Lambda" Conflict):** We identified that the hyperparameter λ, which controls the L1 Pixel Loss, was set too high (λ = 100). The model was being penalized so heavily for pixel deviations that it ignored the PatchGAN's demand for sharpness. It prioritized "mathematical safety" over "visual crispness." This finding led to the design of the final experimental run.

Run 6:

- **Changes from Run 5**
  - **Hyperparameter Tuning (Lambda):**
    - **Change:** Drastically reduced the L1 Loss weight λ from **100** down to **10**.
    - **Reasoning:** In Run 5, the high penalty (λ=100) forced the model to prioritize "pixel safety" (blur) over “realism”.By lowering it to 10, we unleashed the Adversarial Loss, allowing the Discriminator to aggressively punish blurry edges.
  - **Stabilization Technique (Label Smoothing):**
    - **Change:** Implemented **One-Sided Label Smoothing**. Instead of telling the Discriminator that real images are a perfect 1.0, we tell it they are 0.9.
    - **Mechanism:** smooth_real_labels = tf.ones_like(real_output) * 0.9.
    - **Impact:** This prevents the Discriminator from becoming "overconfident" (outputting exactly 1.0 or 0.0), which tends to produce exploding gradients that destabilize the Generator.

- **Goal**
  - To break the "Visual Stagnation" of Run 5 by shifting the model's priority from "Mathematical Safety" (L1) to "Visual Crispness" (Adversarial), while using Label Smoothing to prevent the now-stronger Discriminator from crashing the training.
- **Observation**
  - **Training Duration: ≈ 8 hour and 15 minutes**
  - **Visual Breakthrough:** Some digits became sharper with better defined boundaries than run 5.
  - **Conclusion:** We successfully engineered a solution that balances **Structure** (U-Net), **Texture** (PatchGAN), and **Stability** (via Label Smoothing).



3) Mathematical Comparison (Run 5 vs. Run 6)

- **Generator Loss:** Dropped from ~11.0 to ~2.0. While mostly due to the reduced Lambda (λ=10), this effectively re-scaled the gradients, preventing the L1 term from drowning out the adversarial learning.
- **Discriminator Equilibrium** (Last 20 Epochs Analysis):
    - **Run 5 (~1.23):** calculated over the stable phase (Last 20 Epochs), the average loss was **1.226**. The Discriminator was still successfully identifying fakes with **~54%** confidence.
    - **Run 6 (~1.355):** Calculated over the same stable phase (Last 20 Epochs), the average loss was **1.355**.Discriminator Loss of 1.355 means **50.79%** chance of identifying fakes. If the discriminator is guessing randomly (50% probability), the Binary Cross Entropy loss formula is:

      Loss= - [ln(0.5)+ln(0.5))] ≈ -[-0.693 + -0.693]= 1.386

- **Significance:**
    - **The "Confusion" Goal:** While a difference of 0.15 seems numerically small, in logarithmic loss terms, it is the difference between a "confident" critic (Run 5) and a "confused" critic (Run 6). Run 6 approached a stable adversarial balance, in which the Discriminator’s predictions became statistically indistinguishable from random guessing (≈50% accuracy). While this does not

constitute a provable Nash equilibrium, it indicates that neither network could significantly improve without destabilizing the other

- **Validation of Label Smoothing:** The fact that the loss stabilized at 1.36 (just under the theoretical maximum of 1.386) rather than exploding to infinity proves that our **One-Sided Label Smoothing** worked. It kept the Discriminator healthy enough to provide gradients, but prevented it from overpowering the Generator.
- **Visual Correlation:** The mathematical shift from 1.226 (54% accuracy) to 1.355 (50.79% accuracy) directly correlates with the visual shift from "Blurry" to "Sharp." By removing the obvious tell (blurriness), we forced the Discriminator to resort to guessing, proving the generated images were structurally indistinguishable from the real data.

4) Phase 2 Conclusion: The GAN Journey

Our initial "Regular" Encoder-Decoder model (from **phase 1**) succeeded in guessing the correct numbers but failed to produce convincing images.It produced "blurry averages" to minimize error, resulting in digits that lacked definition

The Solution (Phase 2 Summary):

We hypothesized that a Generative Adversarial Network (GAN) could solve this by introducing a "Critic" (Discriminator) that penalizes blurriness. Our experimentation followed a clear engineering trajectory:

- **Geometric Feasibility:** We first solved mechanical artifacts (Checkerboards) by replacing Transposed Convolutions with UpSampling layers (Run 1 -> Run 2).
- **Structural Integrity:** We solved "melting" shapes by implementing a U-Net architecture with skip connections, allowing the model to understand geometry (Run 2 -> Run 3).

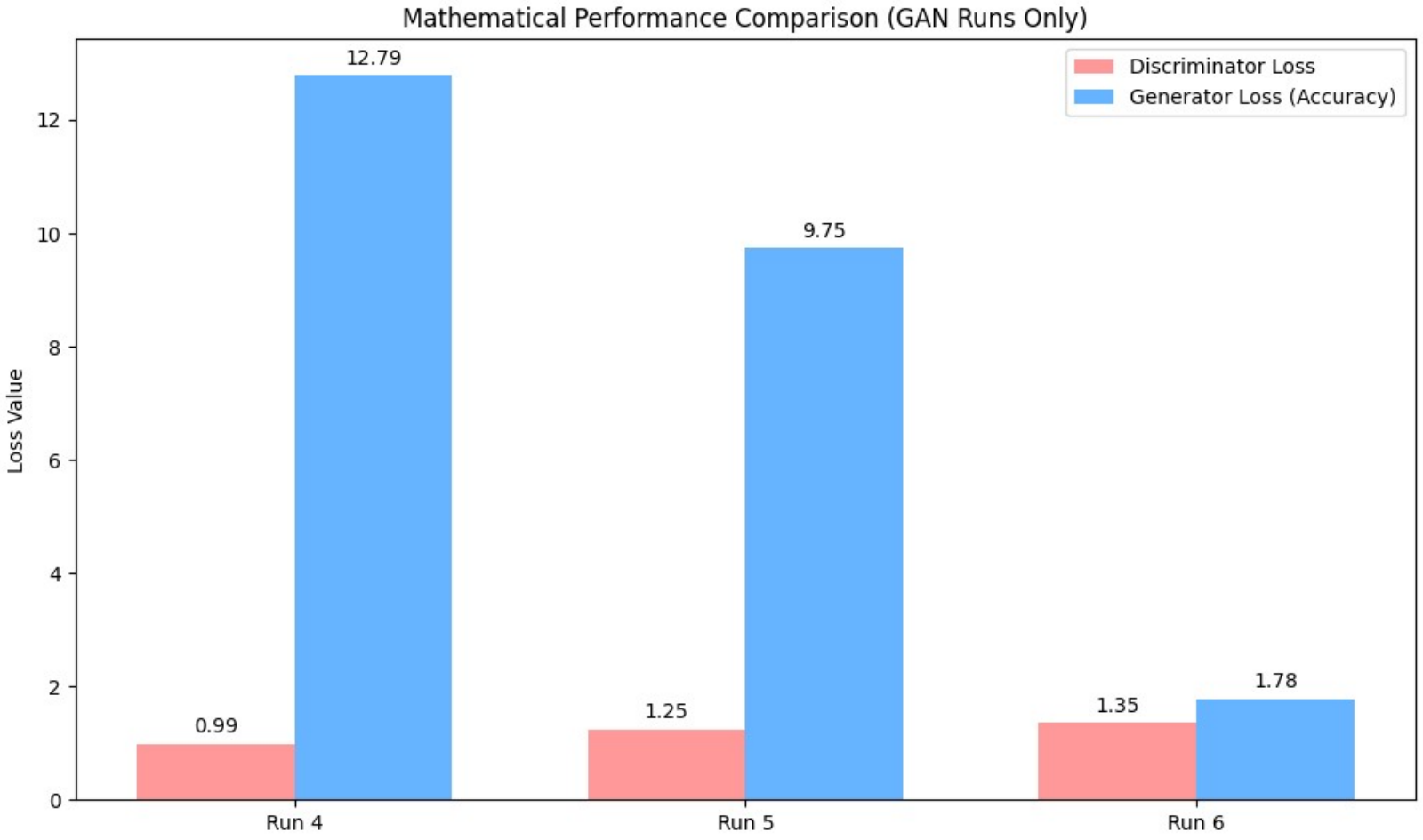
- **Adversarial Balance:** We solved instability and stagnation by upgrading to a **Texture-Aware PatchGAN** and tuning the L1 Loss weight λ from 100 down to 10 (Run 5 -> Run 6).

Final Verdict:

Run 6 stands as the definitive success of this project. It achieved the closest stable adversarial balance observed in our experiments (Discriminator Loss $\approx$ 1.355) alongside high visual fidelity. These results demonstrate that, for this image-to-image translation task, the adversarial approach is substantially more effective than standard pixel-wise regression. While the GAN was significantly harder to stabilize, the final generated digits exhibit sharp, high-contrast structure that the regression-based model consistently failed to produce under pixel-wise loss formulations.

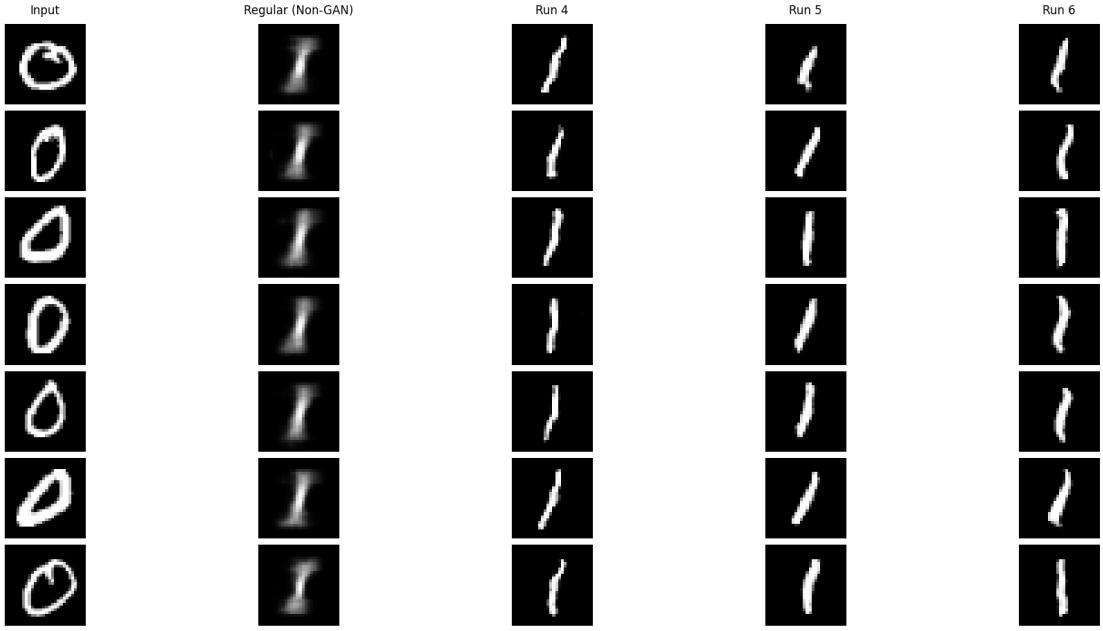
Visualization (new compare_models.py):

- **Mathematical Comparison (Loss Dynamics):**
  - **Scope:** We isolated the GAN models (**Runs 4, 5, and 6**) because their loss functions are comparable, whereas the Regular model uses a different metric.

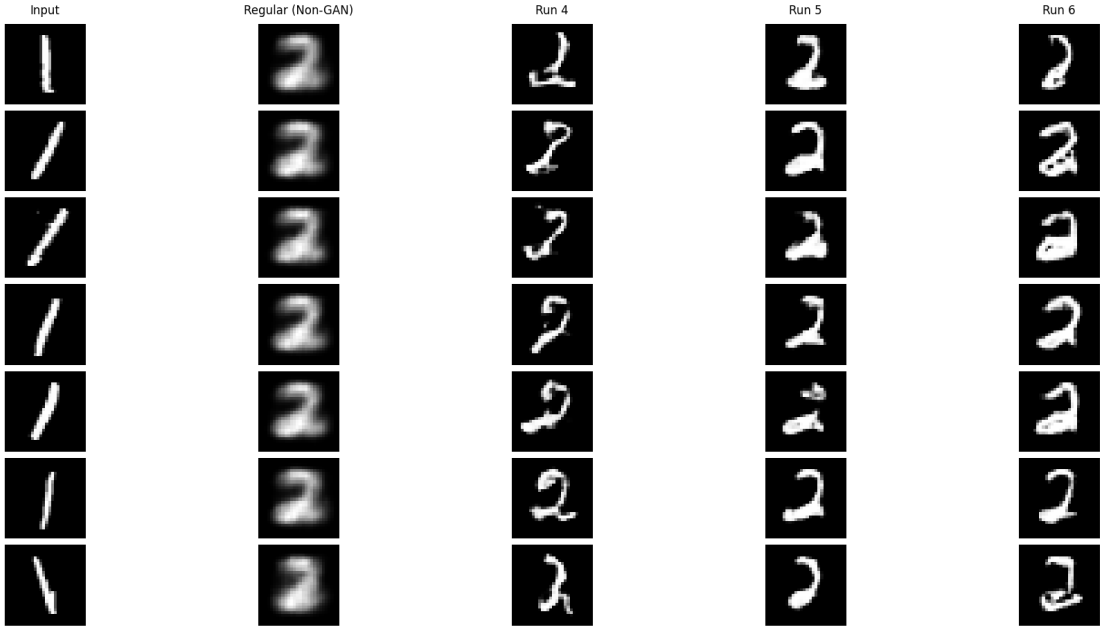


- **Visual Evolution (Structure & Texture):**
  - **Scope:** We compared the **"Regular (Non-GAN)"** baseline against the three final GAN models (**from Run 4, Run 5, Run 6**).

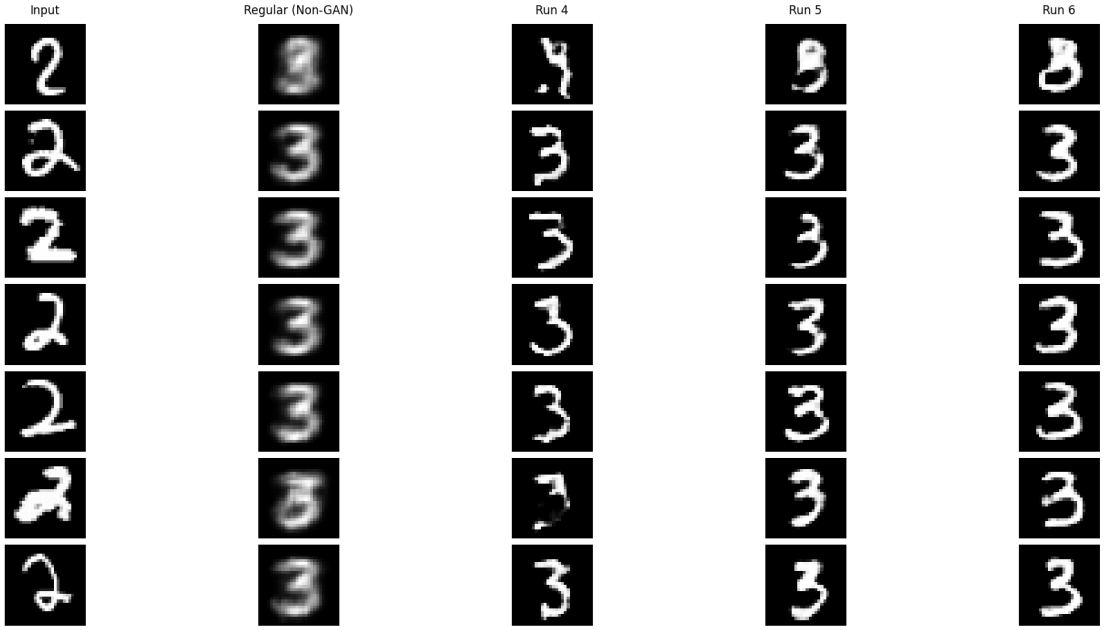
Evolution of Digit 0 -> 1



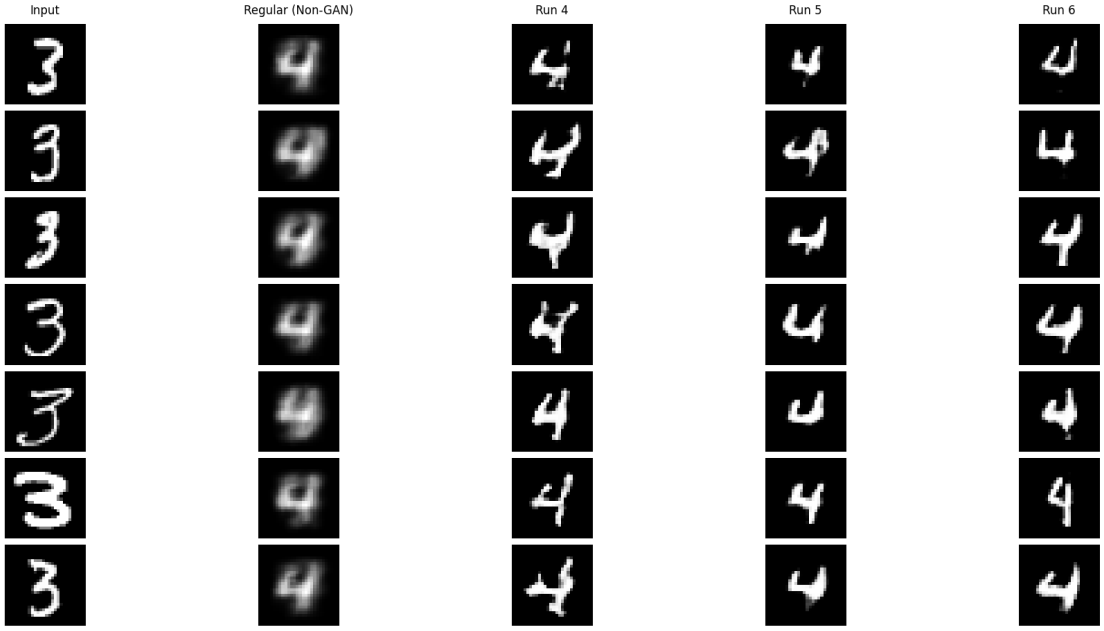
Evolution of Digit 1 -> 2



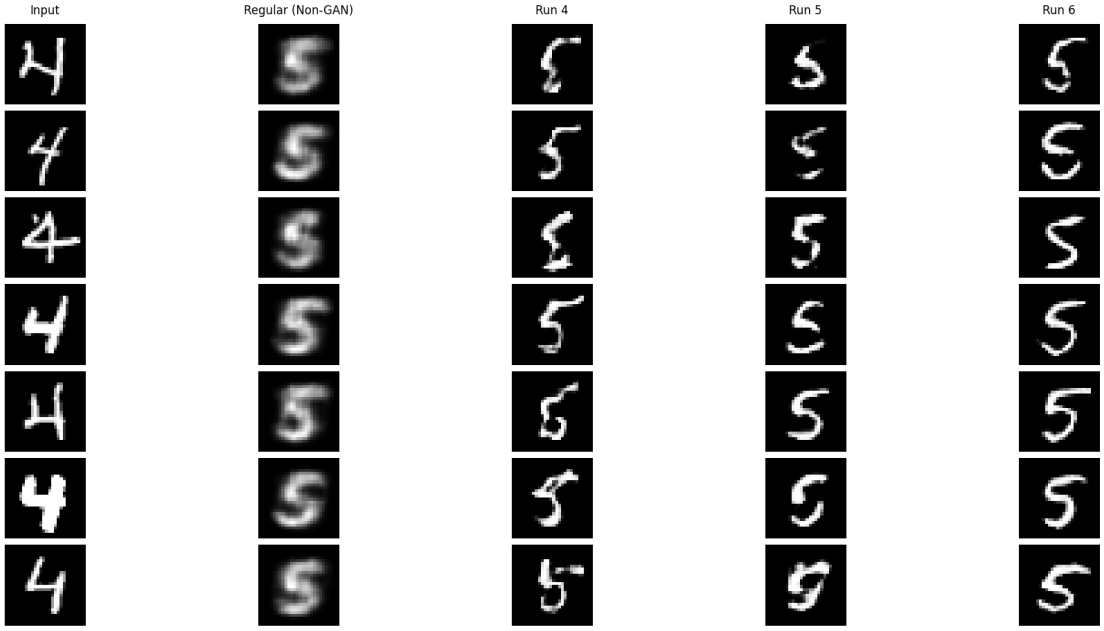
Evolution of Digit 2 -> 3



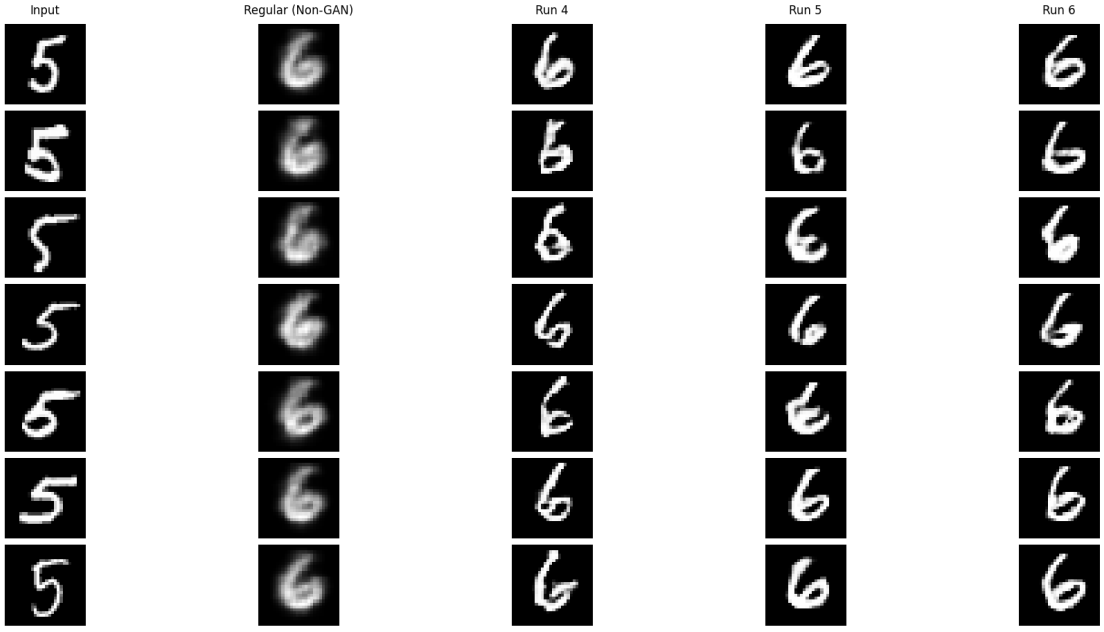
Evolution of Digit 3 -> 4



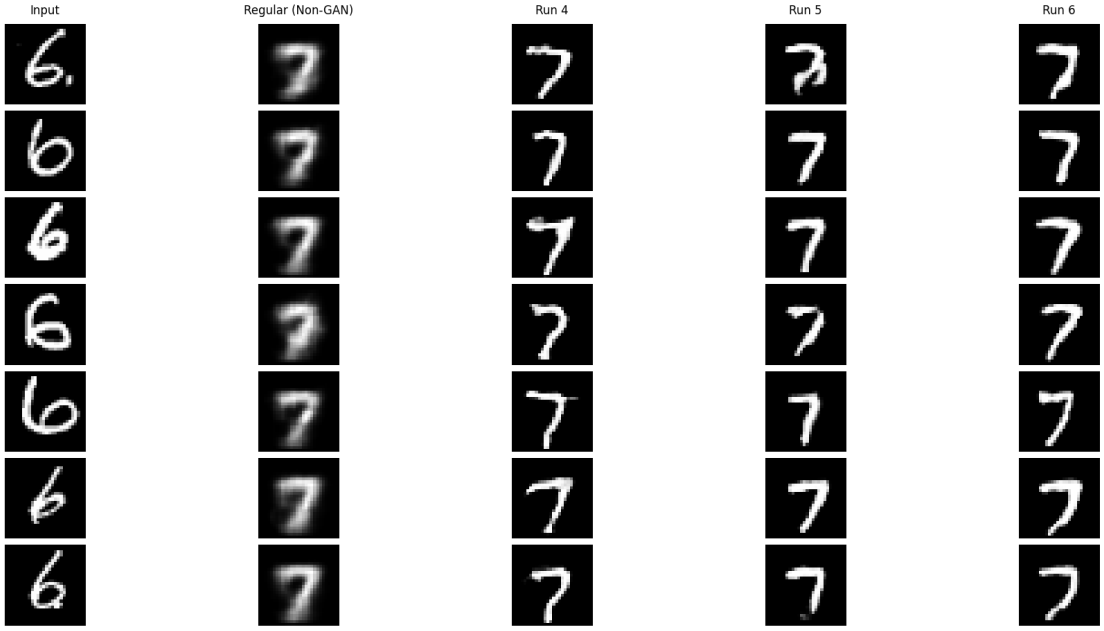
Evolution of Digit 4 -> 5



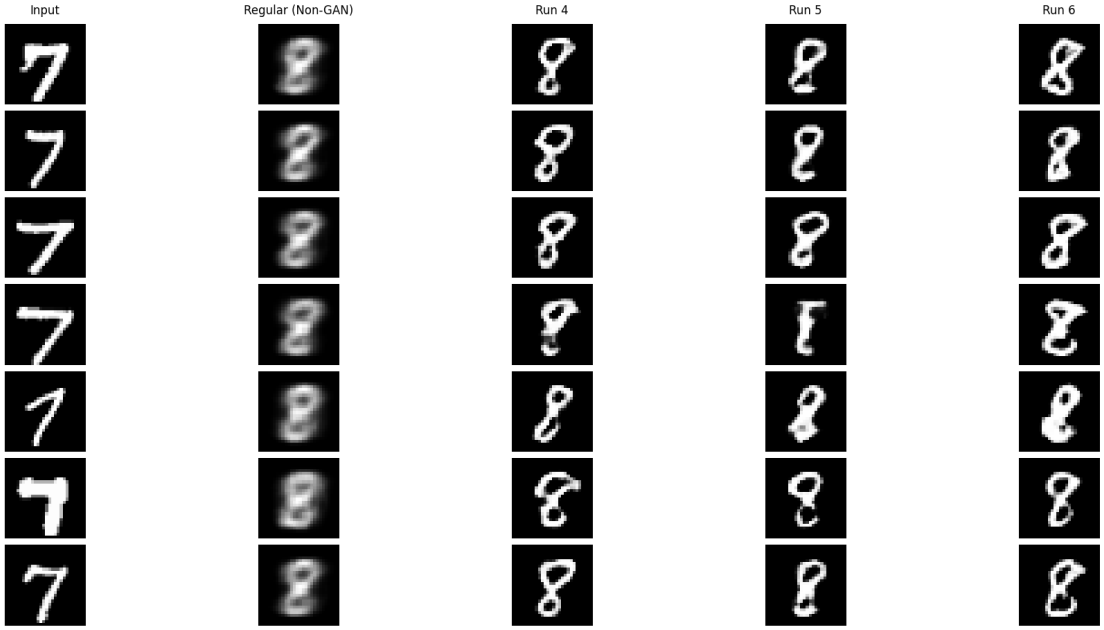
Evolution of Digit 5 -> 6



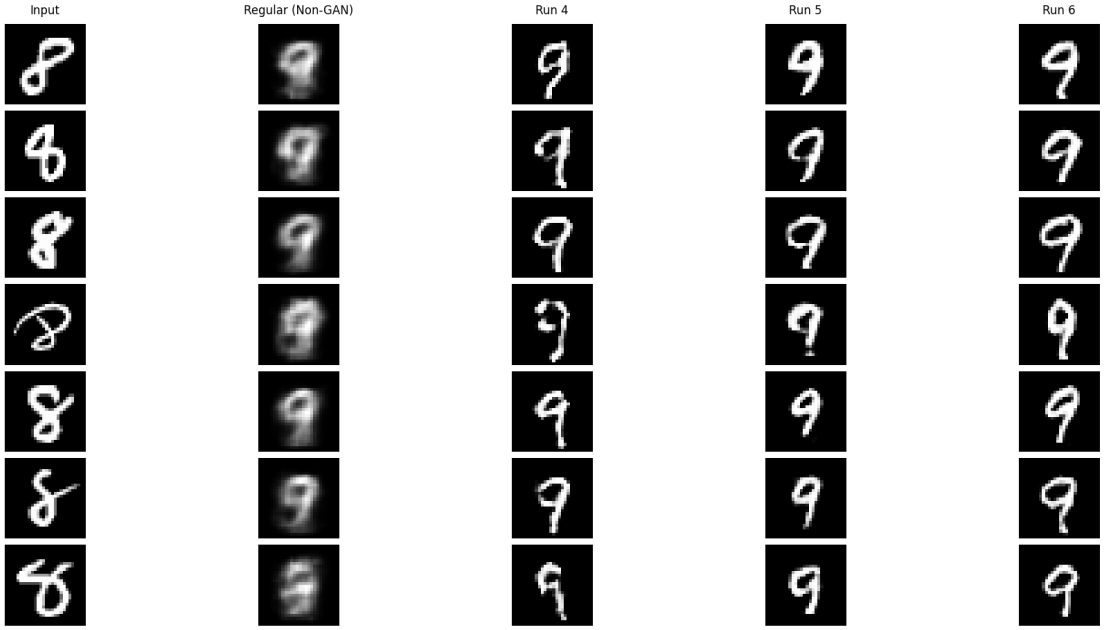
Evolution of Digit 6 -> 7



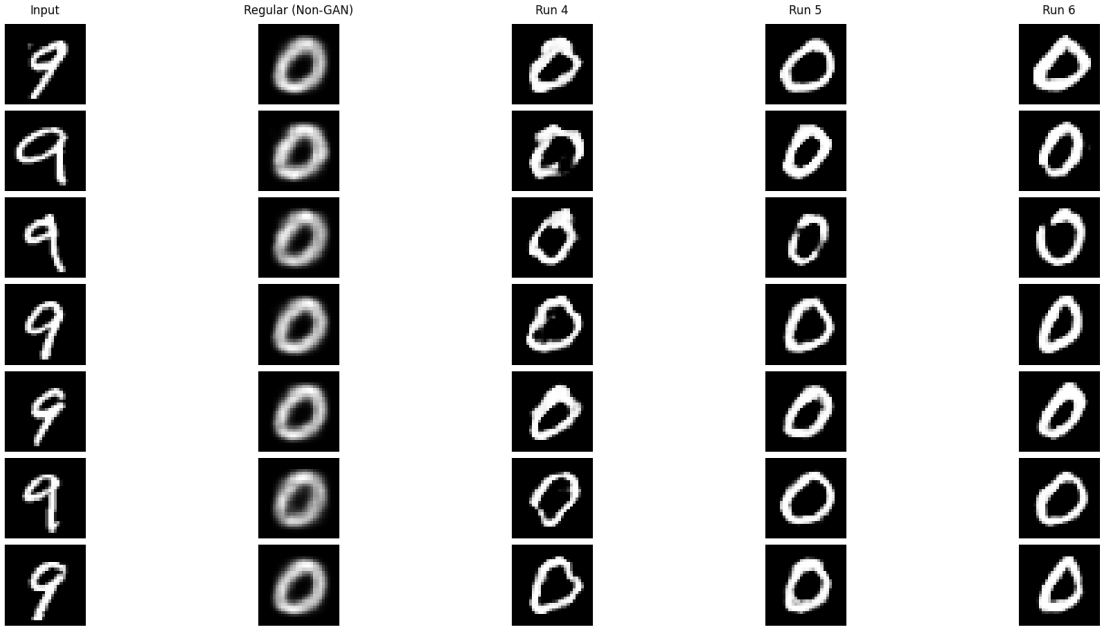
Evolution of Digit 7 -> 8



Evolution of Digit 8 -> 9



Evolution of Digit 9 -> 0



4) Validation & Benchmarking

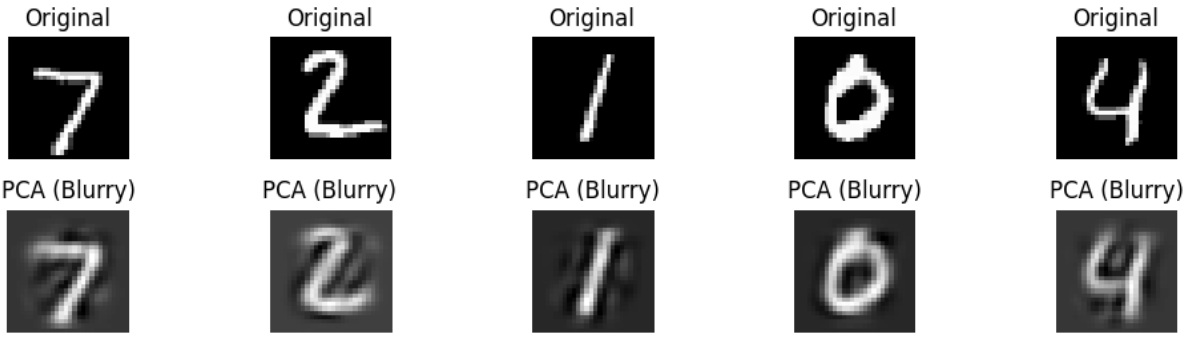
To satisfy the assignment requirements, we performed two final validation tests using an external script (final_validation.py).

A. The "Turing Test" (Classifier Accuracy) We trained a standard CNN Classifier and used it to judge the images generated by our GAN (Run 6).

- **Method:** We fed the GAN inputs (e.g., "3") and asked the Classifier to identify the output. If the GAN successfully generated a legible "4", the Classifier would predict "4".
- **Result:** The GAN achieved a recognition accuracy of **96.90%**.
- **Conclusion:** This proves that the generated digits are not just identifiable by the human eye, but contain the mathematically correct features required for machine recognition. The fact that nearly 97% of the generated digits are recognized correctly validates the architectural choices of Phase 2.

B. Comparison with PCA (Principal Component Analysis) As requested, we compared our GAN with a Linear PCA reconstruction.

- **PCA Performance:** When reducing MNIST digits to 32 components (similar to our latent bottleneck) and reconstructing them, PCA yielded a Mean Squared Error (MSE) of **0.0167**.
- **Visual Comparison:** PCA reconstruction resulted in blurry digits.
- **Why GAN Wins:** PCA is strictly an information compression algorithm; it cannot "invent" new data. It cannot transform a "3" into a "4" because there is no linear vector that maps one digit class to another. Our GAN, utilizing non-linear activation functions (ReLU/LeakyReLU) and Adversarial Loss, successfully models the complex, non-linear transformation required for this task.



Areas Where AI Assistance Was Used

For the development of the code, I used Gemini to generate specific blocks of code and answer architectural questions regarding Generative Adversarial Networks. Specifically, I prompted the AI for the following:

Architecture & Model Selection:

- **Generator Design:** I asked how to adapt a standard GAN (which usually takes random noise) to take an input image instead. The AI

suggested a **Conditional GAN (cGAN)** structure and later recommended upgrading to a **U-Net** architecture (with skip connections) when I encountered the "melting shape" issue.

- **Discriminator Design:** I asked for the specific implementation of a **PatchGAN** discriminator. The AI explained the math behind the 7*7 output grid (using strides instead of pooling).

Code Implementation & Syntax I utilized the AI as a high-level coding assistant throughout the project. Specifically:

- **Visualization:** I used the AI to generate all the Python scripts for plotting the loss graphs and creating the side-by-side image comparison grids.
- **Data Pipelines:** It helped me by explaining the specific syntax needed for TensorFlow Datasets, particularly how to properly shuffle, batch, and map the paired images.
- **General Implementation:** The AI generated the core boilerplate code for the training loops and validated the syntax, while also answering theoretical questions I had regarding the stability and mechanics of GANs.

Value of AI Assistance

Without the assistance of the AI, the completion of this assignment would have required significantly more time.

- **Dynamic Documentation:** Writing a custom GAN training loop with GradientTape involves complex TensorFlow syntax that is often poorly documented for specific use cases. The AI replaced hours of searching through documentation by synthesizing the correct syntax instantly.
- **Focus on Concepts:** This efficiency allowed me to spend less time debugging dimension errors and more time focusing on the experiments themselves, analyzing the trade-offs between texture and structure and refining the architecture from Run 1 through Run 6.

Conclusion on AI Usage: The AI acted as a powerful coding assistant that accelerated the development process.